Examples

```
1:  function SelectionSort(A[],n)
2:      if n < 1 then
3:          return
4:      end if
5:      for i = 1 to n − 1 do
6:          if A[i] > A[n] then
7:              Swap(A[i],A[n])
8:          end if
9:      end for
10:     SelectionSort(A[],n − 1)
11: end function
```

Let $T(n)$ be the running time of `SelectionSort` on an input of size n. The loop on lines 5-9 takes $c(n-1)$ time and then we make a recursive call to `SelectionSort` on line 10. By definition the running time of that recursive call is the size of the input, $n-1$. Additionally, if $n=0$ then the code immediately terminates (this is the base case for our recursion) and therefore $T(0)=c$. Therefore we can express the running time of this code as the recurrence relation

$$T(n) = c(n-1) + T(n-1), \qquad T(0) = c$$

We expand this recurrence relation by repeated substitution into this equation. Namely,

$$T(n-1) = c(n-2) + T(n-2)$$

so

$$T(n) = c(n-1) + \underbrace{c(n-2) + T(n-2)}_{\text{From substitution}}.$$

Repeating this process again we have

$$T(n) = c(n-1) + c(n-2) + c(n-3) + T(n-3)$$

We extrapolate the pattern and postulate that for any value of $k \geq 1$

$$T(n) = \sum_{i=1}^{k} c(n-i) + T(n-k).$$

Since we claim this holds for any value of k we choose k such that

$$T(n-k) = T(0) \qquad \Longrightarrow \qquad n-k=0 \qquad \Longrightarrow \qquad k=n.$$

Substituting this value of k into our expression we have

$$T(n) = \sum_{i=1}^{n} c(n-i) + T(0) = \sum_{i=1}^{n-1} c(n-i) + c.$$

We have now reduced the problem of analyzing the running time of this recursive function into simply analyzing

the asymptotic complexity of this summation. Bounding the summation above we have

$$T(n) = \sum_{i=1}^{n-1} c(n-i) + c \leq \sum_{i=1}^{n-1} c(n-1) + c = c(n-1)^2 + c \leq 2cn^2.$$

Bounding the summation below we have

$$T(n) = \sum_{i=1}^{n-1} c(n-i) + c \geq \sum_{i=1}^{n/2} c(n-i) \geq \sum_{i=1}^{n/2} c\left(n - \frac{n}{2}\right) = \frac{cn^2}{4}.$$

Thus,

$$\frac{cn^2}{4} \leq T(n) \leq 2cn^2$$

so

$$\boxed{T(n) \in \Theta(n^2)}$$

Proposition 1. Given the recurrence relation

$$T(n) = c(n-1) + T(n-1), \qquad T(0) = c \tag{1}$$

then for any value of $k \geq 1$

$$T(n) = \sum_{i=1}^{k} c(n-i) + T(n-k).$$

Proof. We prove it by mathematical induction.

Base case When $k = 1$ the recurrence trivially reduces to the given recurrence relation.

Inductive assumption Suppose that $T(n) = \sum_{i=1}^{m} c(n-i) + T(n-m)$ for all values of $m \leq k-1$.

By inductive assumption we have that

$$T(n) = \sum_{i=1}^{k-1} c(n-i) + T(n-(k-1))$$

Substituting $n-(k-1)$ into Equation (1) we have

$$T(n-k+1) = c(n-k) + T(n-k).$$

So,

$$T(n) = \sum_{i=1}^{k-1} c(n-i) + T(n-k+1) = \underbrace{\sum_{i=1}^{k-1} c(n-i) + c(n-k)}_{\sum_{i=1}^{k} c(n-i)} + T(n-k) = \sum_{i=1}^{k} c(n-i) + T(n-k)$$

Therefore, the expression holds for $m = k$ and the result follows by the principle of mathematical induction. $\square$

```
function SelectMax(A[],n)
    if n = 1 then
        return A[1]
    end if
    for i = 1 to ⌊n/2⌋ do
        A[i] =Max(A[i],A[n − i + 1])
    end for
    x =SelectMax(A[],⌈n/2⌉)
    return x
end function
```

Let $T(n)$ be the running time of `SelectMax` with an input of size n. We can express the running time of as the recurrence relation

$$T(n) = \frac{cn}{2} + T\left(\frac{n}{2}\right), \qquad T(1) = c,$$

We can simplify this expression if we define $c_1 = \frac{c}{2}$, then

$$T(n) = c_1 n + T\left(\frac{n}{2}\right), \qquad T(1) = c.$$

Performing several substitutions we have

$$\begin{aligned} T(n) &= c_1 n + T\left(\frac{n}{2}\right) \\ &= c_1 n + \frac{c_1 n}{2} + T\left(\frac{n}{4}\right) \\ &= c_1 n + \frac{c_1 n}{2} + \frac{c_1 n}{4} + T\left(\frac{n}{8}\right) \\ &= \sum_{i=0}^{k-1} \frac{c_1 n}{2^i} + T\left(\frac{n}{2^k}\right) \end{aligned}$$

Since this expression holds for any value of k we choose k such that

$$\frac{n}{2^k} = 1 \qquad \Longrightarrow \qquad k = \log_2(n).$$

Now,

$$T(n) = c_1 n \sum_{i=0}^{\log_2(n)-1} \frac{1}{2^i} + T(1) = c_1 n \sum_{i=0}^{\log_2(n)-1} \left(\frac{1}{2}\right)^i + c$$

The summation is a decreasing geometric summation which we bound above by an infinite geometric series,

$$T(n) = c_1 n \sum_{i=0}^{\log_2(n)-1} \left(\frac{1}{2}\right)^i + c \leq c_1 n \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i + c = 2c_1 n + c \leq (2c_1 + c)n$$

We trivially bound $T(n)$ from below using the original recurrence relation,

$$T(n) = c_1 n + T\left(\frac{n}{2}\right) \geq c_1 n$$

Therfore,

$$\boxed{T(n) \in \Theta(n).}$$

```
1: function BinarySearchRec(A[],i,j,K)
2:     if i ≤ j then
3:         midp = ⌊(i + j)/2⌋
4:         if K = A[midp] then
5:             index = midp
6:         else if K < A[midp] then
7:             index =BinarySearchRec(A[],i,midp − 1,K)
8:         else
9:             index =BinarySearchRec(A[],midp + 1,j,K)
10:        end if
11:        return index
12:    end if
13: end function
```

We begin by observing that the best case running time is constant (this occurs when $K = A[midp]$). Because of this fact we want to check whether the worst case running time is different than the best case. Let $T(n)$ be the **worst case** running time of `BinarySearchRec` with input of size $n = j - i + 1$. Note that the size of the recursive call may vary depending upon the contents of the array;

If line 7 executes The size of the recursive call on line 7 is approximately $midp - i = \frac{i+j}{2} - i = \frac{j-i}{2} = \frac{n-1}{2}$.

If line 9 executes The size of the recursive call on line 9 is approximately $j - midp = j - \frac{i+j}{2} = \frac{j-i}{2} = \frac{n-1}{2}$.

It turns out that ignoring the floor function used when computing $midp$ has actually resulted in a more frustrating recurrence relation to analyze (a recursive call of size $\frac{n-1}{2}$ may result in some frustrating algebra). Therefore, we shall analyze the parity of $i + j$ and use this to determine the actual sizes of the recursive calls.

If $(i + j)$ is even If $(i + j)$ is even, then $midp = \frac{i+j}{2}$ and we always make a recursive call of size $\frac{n-1}{2}$.

If $(i + j)$ is odd If $(i + j)$ is odd, then $midp = \frac{i+j-1}{2}$. The size of the recursive calls in this case are

If line 7 executes The size of the recursive call on line 7 is $midp - i = \frac{i+j-1}{2} - i = \frac{j-i-1}{2} = \frac{n-2}{2}$.

If line 9 executes The size of the recursive call on line 9 is $j - midp = j - \frac{i+j-1}{2} = \frac{j-i+1}{2} = \frac{n}{2}$.

Since we are analyzing the **worst case** running time we shall assume that the size of the recursive call is *always* $\frac{n}{2}$ which makes our recurrence relation

$$T(n) = c + T\left(\frac{n}{2}\right), \qquad T(1) = c.$$

Performing several substitutions we have

$$\begin{aligned} T(n) &= c + T\left(\frac{n}{2}\right) \\ &= c + c + T\left(\frac{n}{2^2}\right) \\ &= c + c + c + T\left(\frac{n}{2^3}\right) \\ &= kc + T\left(\frac{n}{2^k}\right) \end{aligned}$$

Since this expression holds for any value of k we choose k such that

$$\frac{n}{2^k} = 1 \qquad k = \log_2(n)$$

Therefore,

$$T(n) = c\log_2(n) + T(1) = c\log_2(n) + c.$$

Thus,

$$\boxed{T(n) \in \Theta(\log(n)).}$$

```
function Fib(n)
    if n = 0 then
        return 0
    end if
    if n = 1 then
        return 1
    end if
    x = Fib(n − 1)
    y = Fib(n − 2)
    return x + y
end function
```

Let $T(n)$ be the running time of `Fib` with an input of size n. We can express $T(n)$ as the recurrence relation

$$T(n) = c + T(n-1) + T(n-2), \qquad T(0) = c \text{ and } T(1) = c.$$

Analyzing this expression using our substitution method may result in a combinatorial explosion in the number of recursive calls (feel free to try this!). Because of this fact, we shall attempt to bound this expression above and below. Observe that the running time is an increasing function of n and therefore $T(n-2) \leq T(n-1)$. So,

$$T(n) = c + T(n-1) + T(n-2) \leq c + T(n-1) + T(n-1) \leq c + 2T(n-1).$$

We shall now perform substitution on this expression,

$$\begin{aligned}
T(n) &\leq c + 2T(n-1) \\
&\leq c + 2\left[c + 2T(n-2)\right] = c + 2c + 2^2T(n-2) \\
&\leq c + 2c + 2^2\left[c + 2T(n-3)\right] = c + 2c + 4c + 2^3T(n-3) \\
&\leq \sum_{i=0}^{k-1} c2^i + 2^kT(n-k)
\end{aligned}$$

Since the size of the recursive call is reduced by one each step, the base case that will always be reached in $T(1) = c$; therefore we choose k such that

$$n - k = 1 \qquad \Longrightarrow \qquad k = n - 1$$

Therefore,

$$T(n) \leq \sum_{i=0}^{n-2} c2^i + 2^{n-1}T(1) = c\sum_{i=0}^{n-2} 2^i + \frac{c}{2}2^n = c(2^{n-1} - 1) + \frac{c}{2}2^n \leq c2^n$$

We note that this is an *exponential* upper bound, should the asymptotic complexity turn out to be exponential, then that is effectively a death sentence for our algorithm. Many people underestimate the growth of exponential functions, but (for example), $2^{150} \approx 1.43 \times 10^{45}$ and the universe has existed for less than 4.5×10^{17} seconds. Therefore, we hope that we were "too aggressive" with our upper bound, otherwise our algorithm is dead on arrival. We shall perform a similar technique to lower bound this function – again since the function is increasing

in its argument, $T(n-1) \geq T(n-2)$, so

$$T(n) = c + T(n-1) + T(n-2) \geq c + 2T(n-2)$$

And as before we perform several substitutions with this expression,

$$\begin{aligned}
T(n) &\geq c + 2T(n-2) \\
&\geq c + 2\left[c + 2T(n-4)\right] = c + 2c + 2^2T(n-4) \\
&\geq c + 2c + 2^2\left[c + 2T(n-6)\right] = c + 2c + 4c + 2^3T(n-6) \\
&\geq \sum_{i=0}^{k-1} c2^i + 2^kT(n-2k)
\end{aligned}$$

Since this expression holds for any value of k we choose k such that

$$n - 2k = 0 \qquad \Longrightarrow \qquad k = \frac{n}{2},$$

Substituting this value we have

$$T(n) \geq \sum_{i=0}^{\frac{n}{2}-1} c2^i + 2^{n/2}T(0) \geq c\sqrt{2}^n.$$

Therefore, $T(n) \in \Omega(\sqrt{2}^n)$. As we mentioned before an exponential lower bound would mean this algorithm is completely unworthy of consideration since even very small input sizes would result in the algorithm taking longer than the universe has existed to terminate. Since that is a long time (citation needed) if we ever obtain an exponential lower bound we shall consider our analysis completed.

```
function DefNotFib2(n)
    if n = 0 then
        return 0
    end if
    if n = 1 then
        return 1
    end if
    x = 0
    for i = 1 to n − 1 do
        x = x+DefNotFib2(i)
    end for
    return x
end function
```

Let $T(n)$ be the running time of `DefNotFib2` with input of size n. We express the running time as the recurrence relation

$$T(n) = c + T(1) + T(2) + \cdots + T(n-2) + T(n-1).$$

Since this recurrence relation looks like the recurrence relation for `Fib` we shall bound it below in a similar fashion,

$$T(n) = c + T(1) + T(2) + \cdots + T(n-2) + T(n-1) \geq c + 2T(n-2) \geq 2T(n-2)$$

and then perform substitutions

$$\begin{aligned} T(n) &\geq 2T(n-2) \\ &\geq 2^2 T(n-4) \\ &\geq 2^3 T(n-6) \\ &\geq 2^k T(n-2k). \end{aligned}$$

We choose k such that

$$n - 2k = 0 \qquad \Longrightarrow \qquad k = \frac{n}{2}$$

which means that

$$T(n) \geq 2^{n/2} T(0) = c2^{n/2}.$$

Therefore,

$$\boxed{T(n) \in \Omega\left(\sqrt{2}^n\right).}$$

```
1: function MergeSort(A[],i,j)
2:     if i < j then
3:         midp = ⌊(i + j)/2⌋
4:         MergeSort(A[],i,midp)
5:         MergeSort(A[],midp + 1,j)
6:         Merge(A[],i,midp,j)
7:     end if
8: end function
```

```
1: function Copy(A[],i,j,B[])
2:     p = 1
3:     for k = i to j do
4:         B[p] = A[k]
5:         p = p + 1
6:     end for
7:     B[p] = ∞
8: end function
```

```
1: function Merge(A[],first,midp,last)
2:     Copy(A[],first,midp,L[])
3:     Copy(A[],midp + 1,last,R[])
4:     i = 1
5:     j = 1
6:     for k = first to last do
7:         if L[i] < R[j] then
8:             A[k] = L[i]
9:             i = i + 1
10:        else
11:            A[k] = R[j]
12:            j = j + 1
13:        end if
14:    end for
15: end function
```

We begin analyzing the running time of `Copy` and `Merge`. Let $n = j - i + 1$, then `Copy` takes cn time and for analyzing `Merge` let $n = last - first + 1$, then lines 2-3 of `Merge` take cn time and the for loop on lines 6-14 takes cn time which means `Merge` is in $\Theta(n)$. Now, let us analyze the running time of `MergeSort`; let $T(n)$ be the running time of `MergeSort` on input of size $n = j - i + 1$.

The size of the recursive call on line 4 is $midp - i + 1 = \frac{i+j}{2} - i + 1 = \frac{j-i+2}{2} \approx \frac{n}{2}$ and the size of the recursive call on line 5 is $j - (midp + 1) + 1 = j - \frac{i+j}{2} = \frac{j-i}{2} \approx \frac{n}{2}$ which means that

$$T(n) = cn + 2T\left(\frac{n}{2}\right), \qquad T(1) = c.$$

We perform several substitutions,

$$\begin{aligned}
T(n) &= cn + 2T\left(\frac{n}{2}\right) \\
&= cn + 2\left[\frac{cn}{2} + 2T\left(\frac{n}{2^2}\right)\right] = 2cn + 2^2T\left(\frac{n}{2^2}\right) \\
&= 2cn + 2^2\left[\frac{cn}{2^2} + 2T\left(\frac{n}{2^3}\right)\right] = 3cn + 2^3T\left(\frac{n}{2^3}\right) \\
&= kcn + 2^kT\left(\frac{n}{2^k}\right)
\end{aligned}$$

We choose k such that

$$\frac{n}{2^k} = 1 \qquad \Longrightarrow \qquad k = \log_2(n).$$

Therefore,

$$T(n) = cn\log_2(n) + 2^{\log_2(n)}T(1) = cn\log_2(n) + cn.$$

Thus,

$$\boxed{T(n) \in \Theta(n\log(n)).}$$

```
1:  function MergeSort2(A[],i,j)
2:      if i < j then
3:          n = j − i + 1
4:          m_1 = i + ⌊n/3⌋
5:          m_2 = i + ⌊2n/3⌋
6:          MergeSort2(A[],i,m_1)
7:          MergeSort2(A[],m_1 + 1,m_2)
8:          MergeSort2(A[],m_2 + 1,j)
9:          Merge(A[],i,m_1,m_2)
10:         Merge(A[],i,m_2,j)
11:     end if
12: end function
```

Let $T(n)$ be the running time of `MergeSort2` on input of size $n = j - i + 1$. We leave it to the reader to show that the size of the recursive calls on line 6,7, and 8 are all approximately $\frac{n}{3}$ and the the running time of the two calls to merge on lines 9 and 10 is cn. Assuming the reader has done these calculations, they should agree that the recurrence relation is

$$T(n) = cn + 3T\left(\frac{n}{3}\right), \qquad T(1) = c$$

We perform several substitutions,

$$\begin{aligned}
T(n) &= cn + 3T\left(\frac{n}{3}\right) \\
&= cn + 3\left[\frac{cn}{3} + 3T\left(\frac{n}{3^2}\right)\right] = 2cn + 3^2T\left(\frac{n}{3^2}\right) \\
&= 2cn + 3^2\left[\frac{cn}{3^2} + 3T\left(\frac{n}{3^3}\right)\right] = 3cn + 3^3T\left(\frac{n}{3^3}\right) \\
&= kcn + 3^kT\left(\frac{n}{3^k}\right)
\end{aligned}$$

We choose k such that

$$\frac{n}{3^k} = 1 \qquad \Longrightarrow \qquad k = \log_3(n).$$

Therefore,

$$T(n) = cn\log_3(n) + 3^{\log_3(n)}T(1) = cn\log_3(n) + cn.$$

Thus,

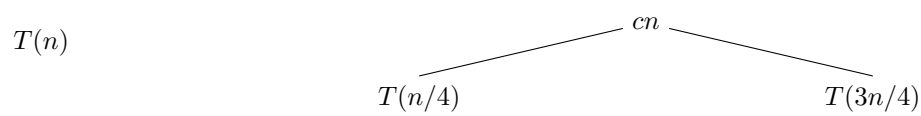
$$\boxed{T(n) \in \Theta(n\log(n)).}$$

```
1: function MergeSort3(A[],i,j)
2:     if i < j then
3:         n = j − i + 1
4:         midp = i + ⌊n/4⌋
5:         MergeSort3(A[],i,midp)
6:         MergeSort3(A[],midp + 1,j)
7:         Merge(A[],i,midp,j)
8:     end if
9: end function
```

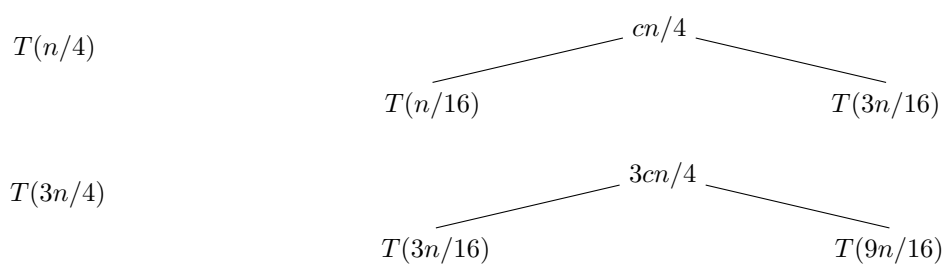
Let $T(n)$ be the running time of `MergeSort3` on input of size $n = j - i + 1$. The size of the recursive call on line 5 is approximately $\frac{n}{4}$ and the size of the recursive call on line 6 is approximately $\frac{3n}{4}$, therefore

$$T(n) = cn + T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right).$$

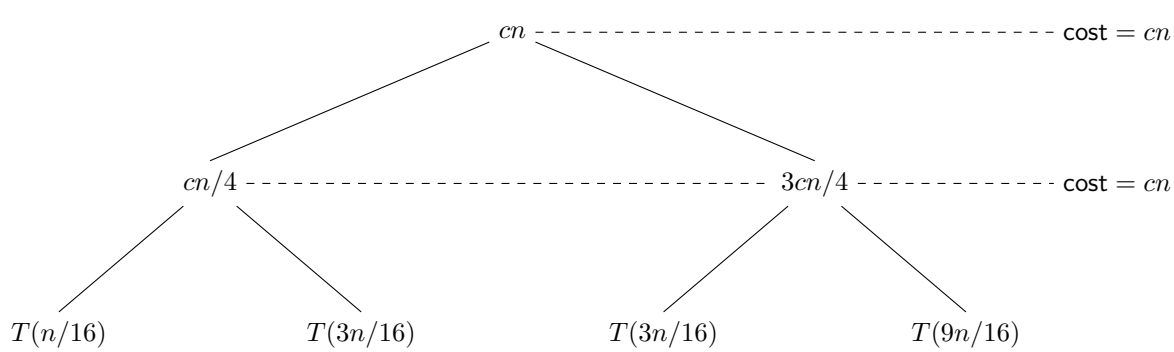
Using our substitution technique to analyze this recurrence relation will again result in a combinatorial explosion in the number of terms, one way to reconcile this is to use a "recursion tree". To construct a recursion tree we separate the problem into the "recursive work" (the calls to $T(n/4)$ and $T(3n/4)$) and the "non-recursive work" (the cn term). We then *visualize* the recurrence relation with a tree



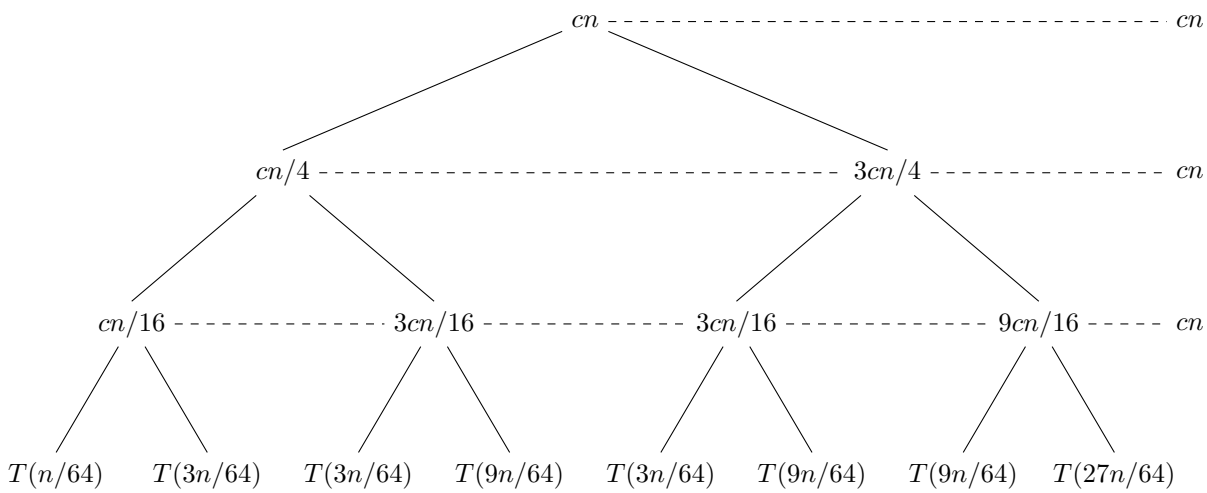
we can then visualize the two recursive calls with their own subtrees,



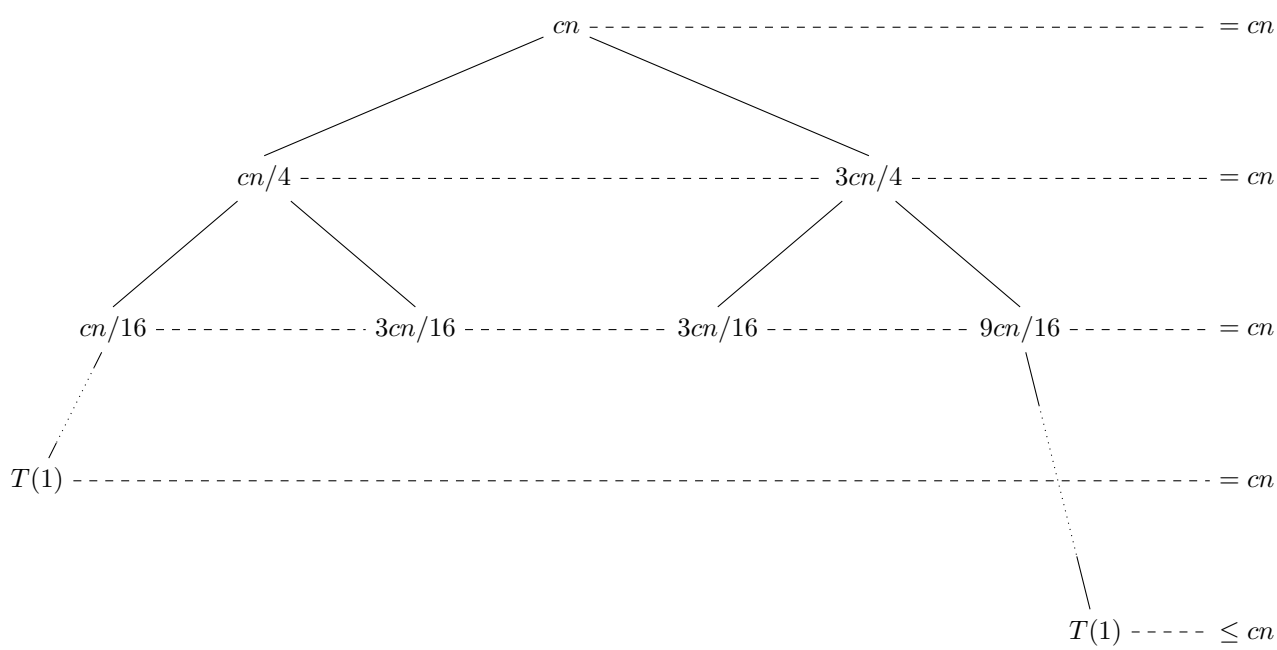
and we can then "substitute" these trees for into the original and we shall begin computing the "cost" at each level of the tree,



If we repeat this process we obtain the tree,



Eventually one of the recursive calls will arrive at the base case; the first branch of the tree of that will do this will be the left branch, because the recursive call of size $n/4$ is smaller than the recursive call of size $3n/4$. Similarly, the last branch of the tree to reach the base case would be the rightmost branch. Furthermore, we observe that the cost at each level appears to be the same, cn. Therefore the tree will eventually have the form



We again note that this is simply a visualization of the substitution process and if compute the sum of the cost at each level, then we would have the running time. Unfortunately, after the first branch terminates it becomes difficult to determine the exact cost at each level, therefore we resort to bounding. We bound it below by summing until the first branch terminates so the question becomes: when does this branch terminate? The size of the recursive call in the leftmost branch is $\frac{n}{4^k}$ and it terminates when

$$\frac{n}{4^k} = 1 \qquad \Longrightarrow \qquad k = \log_4(n)$$

and we can bound the running time below by analyzing the summation

$$T(n) \geq \sum_{i=0}^{\log_4(n)} cn = cn(\log_4(n) + 1)$$

To bound the running time from above we observe that for every level after the first to terminate (the kth level) that some of the paths that were contributing to the pattern have terminated (such as the left most path). Therefore, the cost of each level below the kth is *bounded above* by the pattern we noticed, cn. Secondly, the height of the tree can be computed similarly to the length of the shortest path from the root to a leaf (k). The size of the recursive call in the rightmost branch is $\frac{3^k n}{4^k}$ and it terminates when

$$\frac{3^k n}{4^k} = 1 \qquad \Longrightarrow \qquad k = \log_{\frac{4}{3}}(n)$$

and we can bound the running time above by analyzing the summation

$$T(n) \leq \sum_{i=0}^{\log_{\frac{4}{3}}(n)} cn = cn(\log_{\frac{4}{3}}(n) + 1)$$

Using these two bounds we can now, FINALLY, determine that

$$\boxed{T(n) \in \Theta(n \log(n))}$$

$T(n) = T(n-1) + c$	$T(n) \in \Theta(n)$
$T(n) = T(n-1) + cn$	$T(n) \in \Theta(n^2)$
$T(n) = T(n-1) + cn^2$	$T(n) \in \Theta(n^3)$
$T(n) = T(n/2) + c$	$T(n) \in \Theta(\log(n))$
$T(n) = T(n/2) + cn$	$T(n) \in \Theta(n)$
$T(n) = T(n/2) + cn^2$	$T(n) \in \Theta(n^2)$
$T(n) = T(n/3) + c$	$T(n) \in \Theta(\log(n))$
$T(n) = T(n/3) + cn$	$T(n) \in \Theta(n)$
$T(n) = T(n/3) + cn^2$	$T(n) \in \Theta(n^2)$
$T(n) = 2T(n/2) + cn$	$T(n) \in \Theta(n\log(n))$
$T(n) = 3T(n/3) + cn$	$T(n) \in \Theta(n\log(n))$
$T(n) = T(n/3) + T(2n/3) + cn$	$T(n) \in \Theta(n\log(n))$
$T(n) = T(n/4) + T(3n/4) + cn$	$T(n) \in \Theta(n\log(n))$
$T(n) = T(n/5) + T(4n/5) + cn$	$T(n) \in \Theta(n\log(n))$
$T(n) = T(2n/5) + T(3n/5) + cn$	$T(n) \in \Theta(n\log(n))$
$T(n) = T(n/6) + T(n/3) + T(n/2) + cn$	$T(n) \in \Theta(n\log(n))$
$T(n) = 2T(n-1) + f(n)$	$T(n) \in \Omega(2^n)$
$T(n) = 3T(n-1) + f(n)$	$T(n) \in \Omega(3^n)$
$T(n) = 4T(n-1) + f(n)$	$T(n) \in \Omega(4^n)$
$T(n) = 2T(n-3) + f(n)$	$T(n) \in \Omega(2^{n/3})$
$T(n) = T(n-1) + T(n-2) + f(n)$	$T(n) \in \Omega(2^{n/2})$
$T(n) = T(n-1) + T(n-2) + +T(n-3)f(n)$	$T(n) \in \Omega(2^{n/2})$
$T(n) = \sum_{i=1}^{n-1} T(n-i) + f(n)$	$T(n) \in \Omega(2^{n/2})$

$T(n) = 2T(n/2) + cn^2$ $\qquad$ $T(n) = 4T(n/2) + cn^2$ $\qquad$ $T(n) = 8T(n/2) + cn^2$

Analyzing $T(n) = 2T(n/2) + cn^2$ We begin by performing substitutions

$$\begin{aligned}
T(n) &= cn^2 + 2T\left(\frac{n}{2}\right) \\
&= cn^2 + \frac{cn^2}{2} + 2^2 T\left(\frac{n}{2^2}\right) \\
&= cn^2 + \frac{cn^2}{2} + \frac{cn^2}{2^2} + 2^3 T\left(\frac{n}{2^3}\right) \\
&= \sum_{i=0}^{k-1} \frac{cn^2}{2^i} + 2^k T\left(\frac{n}{2^k}\right)
\end{aligned}$$

We choose k such that

$$\frac{n}{2^k} = 1 \qquad \Longrightarrow \qquad k = \log_2(n)$$

So,

$$T(n) = \sum_{i=0}^{k-1} \frac{cn^2}{2^i} + 2^{\log_2(n)} T(1)$$

We bound the summation to finish our analysis

$$\begin{aligned}
T(n) &= \sum_{i=0}^{k-1} \frac{cn^2}{2^i} + 2^{\log_2(n)} T(1) \\
T(n) &\leq cn^2 \sum_{i=0}^{\infty} \frac{1}{2^i} + cn \leq 2cn^2 + cn \\
T(n) &\geq cn^2
\end{aligned}$$

Thus, $\boxed{T(n) \in \Theta(n^2).}$

Analyzing $T(n) = 4T(n/2) + cn^2$ We begin by performing substitutions

$$\begin{aligned} T(n) &= cn^2 + 4T\left(\frac{n}{2}\right) \\ &= cn^2 + cn^2 + 4^2T\left(\frac{n}{2^2}\right) \\ &= cn^2 + cn^2 + cn^2 + 4^3T\left(\frac{n}{2^3}\right) \\ &= kcn^2 + 4^kT\left(\frac{n}{2^k}\right) \end{aligned}$$

We choose k such that

$$\frac{n}{2^k} = 1 \qquad \Longrightarrow \qquad k = \log_2(n)$$

So,

$$T(n) = cn^2\log_2(n) + 4^{\log_2(n)}T(1)$$

This reduces to

$$T(n) = cn^2\log_2(n) + cn^2$$

Thus, $\boxed{T(n) \in \Theta(n^2\log(n)).}$

Analyzing $T(n) = 8T(n/2) + cn^2$ We begin by performing substitutions

$$\begin{aligned} T(n) &= cn^2 + 8T\left(\frac{n}{2}\right) \\ &= cn^2 + 2cn^2 + 8^2T\left(\frac{n}{2^2}\right) \\ &= cn^2 + 2cn^2 + 2^2cn^2 + 8^3T\left(\frac{n}{2^3}\right) \\ &= \sum_{i=0}^{k-1} 2^icn^2 + 8^kT\left(\frac{n}{2^k}\right) \end{aligned}$$

We choose k such that

$$\frac{n}{2^k} = 1 \qquad \Longrightarrow \qquad k = \log_2(n)$$

So,

$$T(n) = \sum_{i=0}^{k-1} 2^icn^2 + 8^{\log_2(n)}T(1)$$

We analyzing the summation by using the formula for a finite geometric summation

$$T(n) = cn^2\sum_{i=0}^{k-1} 2^i + cn^3 = cn^2(2^k - 1) + cn^3 = cn^2(n-1) + cn^3.$$

Thus, $\boxed{T(n) \in \Theta(n^3)}$